# LAB PROGRAMS

## Question 1 to Question 10

**NAME:RAJASHREE S**

**REG NO:192521079**

## 1. First palindromic string in an array

**Aim:** To implement the above problem in Python using a simple algorithm.

**Algorithm:**

1. Read the required input values.

2. Apply the algorithmic technique specified in the question.

3. Store/update the required intermediate values.

4. Display the final answer.

**Python Code:**

```
n = int(input("Enter number of words: "))
words = []
for i in range(n):
    words.append(input("Enter word: "))
answer = ""
for word in words:
    if word == word[::-1]:
        answer = word
        break
print("First palindromic string:", answer)
```

**Sample Input:**

Mom

Car

boy

**Sample Output:**

First palindrome: mom

```
Enter number of words: 3
Enter word: mom
Enter word: car
Enter word: boy
First palindromic string: mom
```

**Time Complexity**: O(nL)

**Space Complexity**: O(1)

**Result:** Thus, the program was implemented successfully and the required result was obtained.

## 2. Count elements of two arrays that exist in the other array

**Aim:** To implement the above problem in Python using a simple algorithm.

**Algorithm:**

1. Read the required input values.

2. Apply the algorithmic technique specified in the question.

3. Store/update the required intermediate values.

4. Display the final answer.

**Python Code:**

```
a = list(map(int, input("Enter nums1: ").split()))
b = list(map(int, input("Enter nums2: ").split()))
s1, s2 = set(a), set(b)
answer1 = sum(x in s2 for x in a)
answer2 = sum(x in s1 for x in b)
print([answer1, answer2])
```

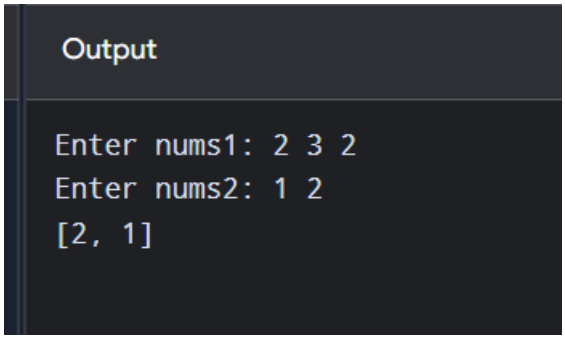
**Sample Input:**

2 3 2

1 2

**Sample Output:**

[2, 1]




**Time Complexity**: O(n+m) average

**Space Complexity**: O(n+m)

**Result:** Thus, the program was implemented successfully and the required result was obtained.

## 3. Sum of squares of distinct counts of all subarrays

**Aim:** To implement the above problem in Python using a simple algorithm.

**Algorithm:**

1. Read the required input values.

2. Apply the algorithmic technique specified in the question.

3. Store/update the required intermediate values.

4. Display the final answer.

**Python Code:**

main.py Visualize

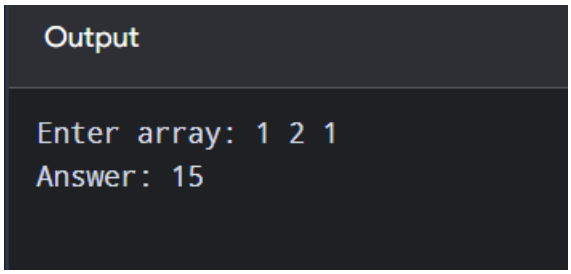
```python
a = list(map(int, input("Enter array: ").split()))
ans = 0
for i in range(len(a)):
 seen = set()
 for j in range(i, len(a)):
  seen.add(a[j])
  ans += len(seen) ** 2
print("Answer:", ans)
```

**Sample Input:**

1 2 1

**Sample Output:**

Answer: 15




**Time Complexity**: $O(n^2)$

**Space Complexity**: O(n)

**Result**: Thus, the program was implemented successfully and the required result was obtained.

## 4. Count equal-value pairs whose index product is divisible by k

**Aim:** To implement the above problem in Python using a simple algorithm.

**Algorithm:**

1. Read the required input values.

2. Apply the algorithmic technique specified in the question.

3. Store/update the required intermediate values.

4. Display the final answer.

**Pseudo code:**

main.py

```
a = list(map(int, input("Enter array: ").split()))
k = int(input("Enter k: "))
ans = 0
for i in range(len(a)):
 for j in range(i + 1, len(a)):
  if a[i] == a[j] and (i * j) % k == 0:
   ans += 1
print("Number of pairs:", ans)
```

**Sample Input:**

3 1 2 2 2 1 3

2

**Sample Output:**

Number of pairs: 4

Output

```
Enter array: 3 1 2 2 2 1 3
Enter k: 2
Number of pairs: 4
```

**Time Complexity:** $O(n^2)$

**Space Complexity**: O(1)

**Result:** Thus, the program was implemented successfully and the required result was obtained.

## 5. Find the required value from the given test cases

**Aim:** To implement the above problem in Python using a simple algorithm.

**Algorithm:**

1. Read the required input values.

2. Apply the algorithmic technique specified in the question.

3. Store/update the required intermediate values.

4. Display the final answer.

**Python Code:**

main.py Visualize Run

```
a = list(map(int, input("Enter array: ").split()))
print("Maximum:", max(a))
```

**Sample Input:**

1 2 3 4 5

**Sample Output:**

Maximum: 5

Output

```
Enter array: 1 2 3 4 5
Maximum: 5
```

**Time Complexity:** O(n)

**Space Complexity**: O(1)

**Result:** Thus, the program was implemented successfully and the required result was obtained.

## 6. Find maximum after efficient sorting

**Aim:** To implement the above problem in Python using a simple algorithm.

**Algorithm:**

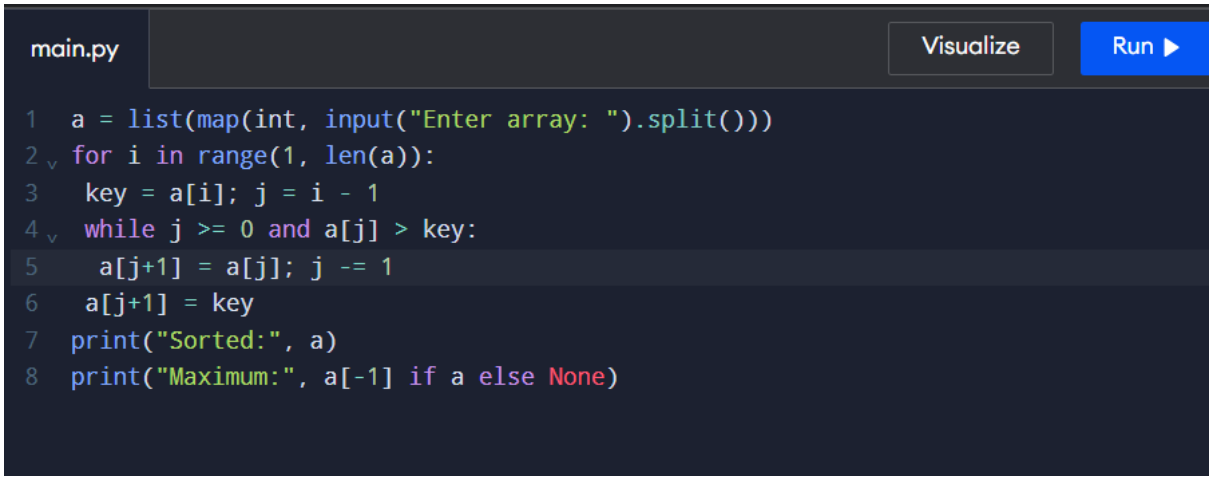
1. Read the required input values.

2. Apply the algorithmic technique specified in the question.

3. Store/update the required intermediate values.

4. Display the final answer.

**Python Code:**




**Sample Input:**

3 7 2 9

**Sample Output:**

Sorted: [2, 3, 7, 9]

```
Output

Enter array: 3 7 2 9
Sorted: [2, 3, 7, 9]
Maximum: 9
```

**Time Complexity:** $O(n^2)$

**Space Complexity**: O(1)

**Result:** Thus, the program was implemented successfully and the required result was obtained.

## 7. Create a list containing unique elements

**Aim:** To implement the above problem in Python using a simple algorithm.

**Algorithm:**

1. Read the required input values.

2. Apply the algorithmic technique specified in the question.

3. Store/update the required intermediate values.

4. Display the final answer.

**Python Code:**

```
main.py                                   Visualize   Run ▶

a = list(map(int, input("Enter array: ").split()))
u = []
for x in a:
  if x not in u:
    u.append(x)
print("Unique list:", u)
```

**Sample Input:**

3 7 3 5 2 5

**Sample Output:**

Unique list: [3, 7, 5, 2]

```
Output

Enter array: 3 7 3 5 2 5
Unique list: [3, 7, 5, 2]
```

**Time Complexity:** $O(n^2)$

**Space Complexity:** O(n)

**Result:** Thus, the program was implemented successfully and the required result was obtained.

## 8. Bubble Sort and Big-O analysis

**Aim:** To implement the above problem in Python using a simple algorithm.

**Algorithm:**

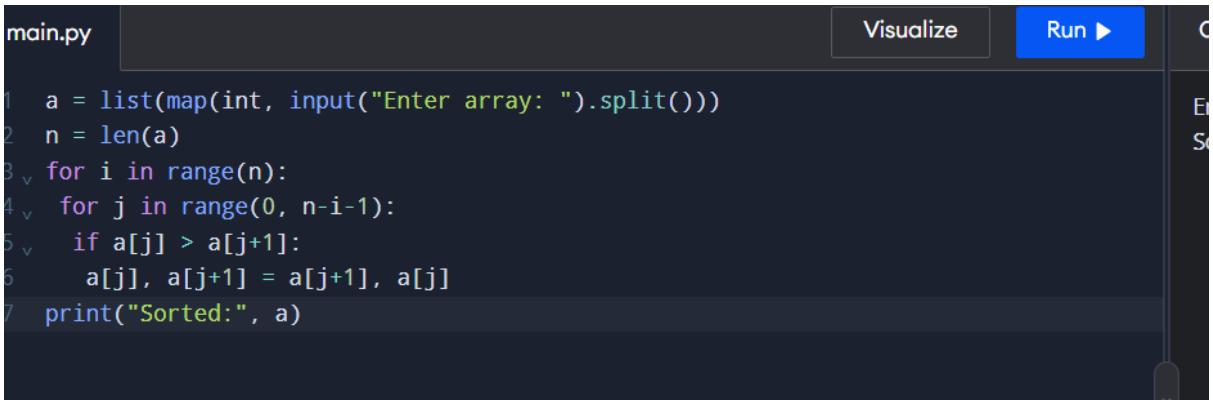
1. Read the required input values.

2. Apply the algorithmic technique specified in the question.

3. Store/update the required intermediate values.

4. Display the final answer.

**Python Code:**

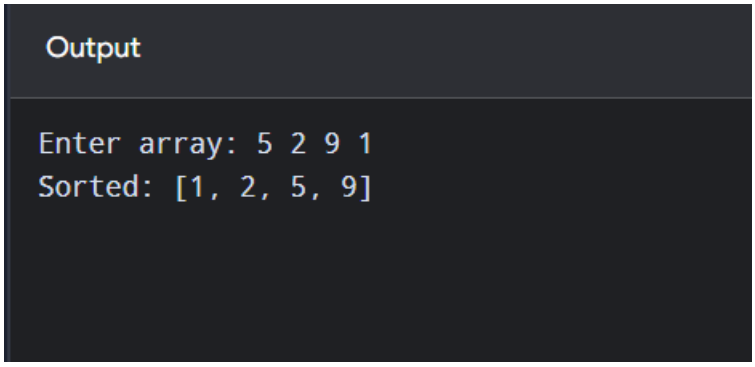



**Sample Input:**

5 2 9 1

**Sample Output:**

Sorted: [1, 2, 5, 9]




**Time Complexity:** $O(n^2)$

**Space Complexity:** O(1)

**Result:** Thus, the program was implemented successfully and the required result was obtained.

# 9. Binary Search in a sorted array

**Aim:** To implement the above problem in Python using a simple algorithm.

**Algorithm:**

1. Read the required input values.

2. Apply the algorithmic technique specified in the question.

3. Store/update the required intermediate values.

4. Display the final answer.

**Python Code:**

main.py Visualize Run ▶

```python
a = list(map(int, input("Enter sorted array: ").split()))
key = int(input("Enter key: "))
lo, hi = 0, len(a)-1
pos = -1
while lo <= hi:
 mid = (lo + hi) // 2
 if a[mid] == key:
  pos = mid; break
 if a[mid] < key: lo = mid + 1
 else: hi = mid - 1
print("Position:", pos + 1 if pos != -1 else -1)
```

**Sample Input:**

3 4 6 8 10

10

**Sample Output:**

Position: 5

Output

```
Enter sorted array: 3 4 6 8 10
Enter key: 10
Position: 5
```

**Time Complexity:** O(log n)

**Space Complexity:** O(1)

**Result:** Thus, the program was implemented successfully and the required result was obtained.

## 10. Sort array in O(n log n) without built-in sorting

**Aim:** To implement the above problem in Python using a simple algorithm.

**Algorithm:**

1. Read the required input values.

2. Apply the algorithmic technique specified in the question.

3. Store/update the required intermediate values.

4. Display the final answer.

**Python Code:**

main.py Visualize

```
def merge_sort(a):
  if len(a) <= 1: return a
  m = len(a)//2
  L, R = merge_sort(a[:m]), merge_sort(a[m:])
  out = []
  i = j = 0
  while i < len(L) and j < len(R):
   if L[i] <= R[j]: out.append(L[i]); i += 1
   else: out.append(R[j]); j += 1
  return out + L[i:] + R[j:]
a = list(map(int, input("Enter array: ").split()))
print("Sorted:", merge_sort(a))
```

**Sample Input:**

5 3 1 4 2

**Sample Output:**

Sorted: [1, 2, 3, 4, 5]

Output

```
Enter array: 5 3 4 1 2
Sorted: [1, 2, 3, 4, 5]
```

**Time Complexity:** O(n log n)

**Space Complexity:** O(n)

**Result:** Thus, the program was implemented successfully and the required result was obtained.